

EXPENSE TRACKER

Project Report & Documentation

Student Name	Gajvelli Naveen
Internship Domain	Python Development
Week	6
Project Type	Python Console Application

GitHub Repository

<https://github.com/gajvellingaveen58-ui/Expense-Tracker>

Task 4: Project Documentation & Presentation

1. Introduction

The Expense Tracker is a Python-based console application developed to help users record and manage daily expenses in a simple and organized way.

The application supports adding, viewing, calculating, summarizing, searching, and deleting expenses. It also demonstrates practical Python and Object-Oriented Programming concepts.

2. Problem Statement

Managing daily expenses manually can be difficult and time-consuming. Users may forget previous expenses or find it difficult to calculate spending by category. This project provides a simple menu-driven solution for recording and managing expense information.

3. Project Objectives

- Develop a simple Python expense management application.
- Allow users to add and store expense records.
- View all recorded expenses and calculate total spending.
- Generate category-wise expense summaries.
- Search expenses by category or description.
- Delete selected expense records.
- Store data permanently using file handling.
- Apply Object-Oriented Programming concepts in a practical project.

4. Major Features

Feature	Description
Add Expense	Accepts amount, category, description, and Personal/Business type with validation.
View Expenses	Displays amount, category, description, and date for every record.
Calculate Total	Calculates total spending.
Category Summary	Groups expenses by category and calculates totals.
Search Expenses	Searches category or description using case-insensitive matching.
Delete Expense	Deletes a selected expense and updates the file.
File Storage	Stores records in expenses.txt.
Error Handling	Handles invalid input and missing/invalid file data.

5. Technologies Used

- **Python:** Main programming language.
- **datetime:** Records expense dates.
- **abc:** Implements abstraction.
- **Text file:** Persistent storage using expenses.txt.

- **Git:** Version control.
- **GitHub:** Repository hosting and submission.

6. Python Concepts Implemented

- Variables and data types: float, strings, lists, and dictionaries.
- Input and output using `input()` and `print()`.
- Conditional statements using `if`, `elif`, and `else`.
- Loops using `while` and `for`.
- Functions and class methods.
- Lists for storing expense objects.
- Dictionaries for category summaries.
- File handling for reading, appending, and rewriting `expenses.txt`.
- Exception handling for `ValueError` and `FileNotFoundError`.
- Date handling using `datetime.now()`.
- **Abstraction:** `ExpenseBase` uses ABC and abstract `display()`.
- **Encapsulation:** `Expense` stores amount in private `__amount` with getter/setter methods.
- **Inheritance:** `PersonalExpense` and `BusinessExpense` inherit from `Expense`.
- **Polymorphism:** Child classes override `display()`.

7. Project Workflow

```
START → Create ExpenseTracker → Load expenses.txt → Display Menu → Add / View / Total / Summary /
Search / Delete / Exit → Return to Menu → EXIT
```

When the program starts, previously saved records are loaded. The application then displays a seven-option menu and continues until the user selects Exit.

8. Important Classes and Functions

- **ExpenseBase:** Defines the abstract `display()` method.
- **Expense:** Stores expense details and provides getter/setter methods.
- **PersonalExpense:** Represents personal expenses and overrides `display()`.
- **BusinessExpense:** Represents business expenses and overrides `display()`.
- **ExpenseTracker:** Manages adding, saving, loading, viewing, calculating, summarizing, searching, and deleting expenses.
- **main():** Starts the application and controls the menu.

9. Screenshots / Output

The project screenshots are organized in the GitHub repository under the screenshots directory:

```
screenshots/
■■■ task-1/
■■■ task-2/
■■■ task-3/
```

These screenshots provide visual evidence of the completed internship tasks and program outputs.

10. Challenges Faced

- Handling invalid amount and menu inputs without crashing.
- Reading and writing expense records correctly using a text file.
- Handling a missing expenses.txt file during first execution.
- Keeping stored data synchronized after deleting an expense.
- Combining abstraction, encapsulation, inheritance, and polymorphism in one project.

11. Future Improvements

- Develop a graphical user interface using Tkinter.
- Replace text-file storage with SQLite or MySQL.
- Add expense editing.
- Add monthly and yearly reports and date filters.
- Add charts and graphs for spending analysis.
- Export reports to CSV or PDF.
- Add user login and multiple-user support.
- Add automated tests.

12. Conclusion

The Expense Tracker successfully demonstrates how Python can be used to build a practical application for managing personal and business expenses.

The project provides features for adding, viewing, searching, calculating, summarizing, and deleting expenses, while using file-based storage to preserve data between executions.

It demonstrates core Python programming and Object-Oriented Programming concepts, including abstraction, encapsulation, inheritance, and polymorphism.

13. GitHub Repository

Complete project source code and screenshots are available at:

<https://github.com/gajvellingaveen58-ui/Expense-Tracker>

14. Project Presentation – Viva Preparation

Question	Answer
What problem does your project solve?	It helps users record and manage expenses instead of maintaining them manually.
How does it work?	It uses a menu-driven console interface and stores records in expenses.txt.
What are the main features?	Add, view, total, category summary, search, delete, and file storage.
Which OOP concepts are used?	Abstraction, encapsulation, inheritance, and polymorphism.
How is encapsulation implemented?	The amount is stored in private <code>__amount</code> and accessed through getter/setter methods.
How is inheritance implemented?	PersonalExpense and BusinessExpense inherit from Expense.
How is polymorphism implemented?	Both child classes override <code>display()</code> .

Question	Answer
How is data stored?	Expense records are stored in expenses.txt.
What was a major challenge?	Combining OOP and file handling while keeping stored data synchronized after deletion.
What is the future scope?	GUI, database storage, charts, reports, editing, exporting, and automated testing.